CURRENT STATUS | 14 OCTOBER 2026, 05:47 UTC

SPO reproduction is still incomplete

Our primary objective is to reproduce the published **Segment Policy Optimization (SPO)** RL result on our A100, starting from the authors' supervised model. Evaluating released trained weights is a useful check, but is not an end-to-end training reproduction. The Dr. GRPO notebook that follows is a **separate historical experiment**, with a different model and benchmark.

Evidence	Result	Meaning
Released SPO weights	56.69%; paper: 56.7%	Reference evaluator agrees
Earlier own run	22.19% at 7,856 updates	Failed before endpoint
Fresh restart	56.85% at 8,016 updates	+17.07 points over SFT
Fixed endpoint	11,040 updates, not reached	No completed reproduction

The fresh test has 11,997 correct of 21,104 sampled answers across all 1,319 GSM8K questions, with 16 answers each. Original SFT scored 8,394 correct: the gain is 17.07 percentage points. All 33,024 saved answers across test, validation and the training subset independently match the native metrics. Validation is 71.67%; training is 86.76% and is not held out. This intermediate test is numerically above paper 56.7%, not a statistical-equivalence claim.

The earlier run's latest test had 4,684 correct answers. It later failed with an optimizer gradient-norm assertion at logged step 7,995, not an allocation timeout. That failure and the preceding accuracy decline need separate explanations.

Paper-level intermediate score, fixed endpoint still ahead

Generation included a start-of-text marker (BOS), but training inputs omitted it. GPU checks found this changes the late model's probabilities strongly. The fresh run corrects this inconsistency and separately handles a verified zero-gradient optimizer boundary. Those diagnostics are not causal proof.

All 51 scheduled scores remain available; none replaces the fixed 11,040-update endpoint and final native test/regrade. The new run has passed the old failure count, but exact historical author-code fidelity and collapse causality remain unresolved. This is **measured RL improvement, not completed reproduction**. It calibrates shared RL practice; it does not establish the best algorithm or prove transfer to recursive decisions, tools, costs or credit assignment.

How to read the historical notebook

The following numbered pages preserve the 7 October Dr. GRPO results and RLM teaching hypotheses. Its small-scale improvements are not a claim that the full paper or SPO training result was reproduced. Current evidence and native receipts are linked from the advisor evidence record.

REPRODUCTION NOTEBOOK | 7 OCTOBER 2026

Learning language-model RL from an official reproduction

We want to establish that our language-model reinforcement learning pipeline does what we think it does: generate answers, assign rewards, change weights, and evaluate those changed weights fairly. We start with the authors' code for *Understanding R1-Zero-Like Training: A Critical Perspective* [1,2]. These historical runs reproduced RL-driven improvement at small scale using its method, not the full published benchmark. Their purpose at the time was to learn how to teach RLMs; the current full-reproduction objective is stated on the preceding status page.

Why do this before recursive language-model RL?

In this experiment, the language model receives a mathematics question and writes one response. A checker compares its final answer with a reference. There are no live Python tools or recursive model calls in these responses. That makes the reward and update easier to inspect.

An RLM adds choices about decomposition, running code, reading intermediate results, and calling other models. A failure can then come from the language model, the surrounding software, the reward, or which actions receive credit. A working ordinary LLM experiment gives us a reference for the shared training machinery. It does not prove that those additional RLM decisions are trained correctly. We will still need separate checks of action boundaries, tool results, costs, and credit assignment.

Pretraining, SFT, and RL are different sources of learning

Stage	What supplies the learning target?
Pretraining	Predicting text from a large existing collection develops the starting language and mathematical abilities.
Supervised fine-tuning (SFT)	Supplied example responses tell the model which tokens to imitate for a prompt.
Reinforcement learning (RL)	The model samples its own responses. Rewards change how likely those responses are under its policy.

Our starting checkpoint is the **Qwen2.5-Math-1.5B base model** [3]. We add no SFT stage. "Base" does not mean untrained: its weights already contain substantial capabilities from earlier training. Calling this R1-Zero-like training does not mean beginning with random weights.

What has been established so far?

Training with the chat prompt improved its full-500 score from **154 to 308**. But the base model already scored **305 with the question alone**. On question-only evaluation, the chat-trained model scored **317**; two fresh question-only training runs scored **314 and 306**. Against 305, their gains of nine and one answer do not demonstrate a stable, material improvement. The longer run improved from **20 to 47 on the same 64-question monitor**. We stopped it by choice to focus on RLM teaching experiments. This demonstrates improvement in that setup, not a repeatable gain beyond the strong question-only baseline. Pages 4–10 preserve the comparisons and examples.

Historical Dr. GRPO cutoff: 7 October 2026, 20:56 UTC. Math training stopped by choice; checkpoints and all completed results retained. Latest progress check: 47/64 at update 192. Lessons for RLMs begin on page 11.

How eight answers produce an update

This page describes the completed short runs. The larger recipe introduced on page 8 changes the training verifier and data budget.

For each training question, the generator samples eight responses at temperature 1. The author reward function extracts the last boxed answer and checks it against the reference. A correct answer earns 1; a wrong or unparseable answer earns 0. There is no separate reward just for using a box. The training actor also assigns 0 to a response stopped by the token limit. The checker evaluates the answer, not the truth of every reasoning step [2].

A teaching example: two successes in a group of eight

Suppose one question produces rewards

$$r = (1, 0, 0, 1, 0, 0, 0, 0), \qquad \bar{r} = 2/8 = 0.25.$$

Dr. GRPO subtracts this group's mean from each reward:

$$A_i = r_i - \bar{r}, \qquad A = (0.75, -0.25, -0.25, 0.75, -0.25, -0.25, -0.25, -0.25).$$

The two successful responses receive positive advantages; the other six receive negative advantages. The update pushes toward the successful sampled responses and away from the unsuccessful ones. If all eight answers are wrong, or all eight are correct, their advantages are all zero. The sum being zero in a mixed group does not cancel the learning signal: different responses have different token probabilities and gradients.

What is special about Dr. GRPO?

Dr. GRPO is the authors' variant of Group Relative Policy Optimization. It does not divide advantages by the group's reward standard deviation. It also sums token losses and divides by a constant response limit, here 3000, instead of dividing each response's loss by its own length. These are the method's two normalization changes [1,2]. Its policy loss uses a clipped probability ratio:

$$q_{it} = \frac{\pi_\theta(y_{it} \mid x_i, y_{i,<t})}{\pi_{\text{old}}(y_{it} \mid x_i, y_{i,<t})}, \qquad L = -\frac{1}{B\,3000} \sum_{i=1}^{B} \sum_{t \in \text{response}_i} \min(q_{it} A_i, \text{clip}(q_{it}, 0.8, 1.2) A_i)\,.$$

Here the policy is the model's next-token distribution, θ is its set of weights, and $B = 128$ responses. The group advantage is shared across all response tokens; this experiment does not determine which individual reasoning step deserves credit. We use zero KL penalty: no extra penalty keeps the updated policy close to a reference model.

How the single-GPU run fits together

One collection contains 16 questions times eight responses: 128 trajectories. The learner processes one trajectory at a time, accumulates 128 gradients, and takes one optimizer update. It then sends the new weights to vLLM for the next collection. We use one pass over each collection, full-model BF16 training, learning rate 10^{-6}, and the author's Adam settings.

The pilot's 64 questions gave four updates; each completed 512-question run gave 32 updates from 4096 sampled responses. Training and generation alternate on one A100 with 40 GB of memory. Putting the generator to sleep frees memory; it does not reset the weights. Likewise, switching a model between training and evaluation modes changes its execution behavior, not its learned weights. The optimizer performs the update.

Freeze the questions and the evaluator

We use the official code at commit `dfca49dd460e`, Oat `0.1.3.post1`, vLLM `0.8.4`, and Qwen model revision `4a83ca6e4526`. Full identifiers, package versions, and content hashes are recorded beside this guide in the research record and data manifest [4].

Purpose	Size	Selection and use
Pilot training	64	First 64 indices after shuffling 12,000 author training rows with Python `random.Random(42)`.
Larger training	512	First 512 indices from the same shuffle. This includes the pilot questions; every larger run restarts from the base weights.
Monitoring	64	Shuffle the author's 500 MATH evaluation rows with seed 142; take the first 64. Reused for progress checks.
Final comparison	128	Take the next 128 from that evaluation shuffle. Disjoint from monitoring; used for the larger run's fixed final comparison.

The 12,000 training rows include 4,501 IDs from the original MATH test split. An arbitrary download of that test split would therefore be a poor choice of evaluation data. We checked the author's supplied training collection against the supplied MATH500 questions and found no exact overlaps after whitespace normalization. This does not rule out paraphrases or exposure during the base model's pretraining.

Keep three kinds of settings separate

Weights. The baseline uses the original base checkpoint. Each trained model uses its prescribed final checkpoint. All three Dr. GRPO runs start separately from the base, not from the pilot or each other's weights.

Generation. Each before/after pair uses the same input format, the same 3000-token response limit, and greedy decoding with one answer per question. We evaluate both the Qwen-Math chat prompt and the question alone; page 5 shows what changes. Training uses temperature 1 for varied samples. Comparing base and trained weights under different prompts would mix prompt effects with weight changes. This is a central warning of the paper [1].

Scoring. The author's training program uses a faster checker. The standalone evaluator uses an additional mathematical-equivalence check (`fast=False`). We apply the same standalone checker to both saved answer sets for every reported comparison, including the separate GRPO follow-up. Source questions and reference answers were checked against the frozen data. We never combine scores from different checkers.

Choose the checkpoint before reading the final score

We retain the prescribed final checkpoint and evaluation, even if an earlier evaluation looks better. On the 64 monitoring questions, the chat-trained run's final evaluation scored 36, versus 20 at baseline; another evaluation after the same 32 updates scored 39. The pilot likewise scored 23 and then 22 with identical saved weights. Greedy GPU decoding was not exactly repeatable. We report this variation without assigning it a specific numerical cause or choosing the higher score.

The initial runs use learner seed 42 and the fresh question-only repeat uses 43. The unchanged author actor chooses a time-based generation seed, so learner seed labels do not make the whole run deterministic. Evaluation repeats test generation variation; fresh training runs test whether a gain survives retraining.

A large prompt effect, an uncertain learning gain

Three separate runs restarted from the original base weights. Each used the same 512 questions, 4096 sampled responses, and 32 full-model BF16 updates in the unchanged author algorithm. One used the chat prompt and learner seed 42; two used questions alone, with learner seeds 42 and 43. These are separate training runs, not multiple evaluations of one trained model [4].

Evaluation input	Starting	Chat-trained seed 42	Question-trained seed 42	Question-trained seed 43
Chat prompt	154/500	308/500	168/500	166/500
Question alone	305/500	317/500	314/500	306/500

Within each row, models are compared under matched evaluation settings. Between rows, the input format changes. All trained columns use prescribed final checkpoints. The question-only follow-ups were chosen after seeing the stronger question-only baseline, so they are exploratory, not new untouched confirmation.

On question-only evaluation, chat-prompt training gains 12 answers (18 gains, six losses). The first question-only training run gains nine (24 gains, 15 losses), and the fresh repeat gains one (18 gains, 17 losses). Both differences are positive, but +1.8 and +0.2 percentage points do not establish a stable, material improvement. The repeat improves nine answers and worsens 17 relative to the first question-trained model, for eight fewer correct answers overall. Under chat evaluation it scores 166, with 25 gains and 13 losses against 154.

What do the different gains mean?

The large gain remains the chat-trained model under chat evaluation, from 30.8% to 61.6%. Against the stronger question-only baseline, the fresh training repeat weakens the case for a dependable benefit from this short recipe. This neither proves zero learning nor identifies newly acquired mathematics. It also does not establish that all gains are answer-format repair, or validate RLM tool use and credit assignment across multiple calls.

The chat-trained model reduced capped chat responses from 205 to 29; the two question-trained models still had 201 and 198 in that format. With question-only evaluation, the base had 15 capped responses, the first question-trained model 19, and its fresh repeat 14. Their mean lengths were 694.31, 693.61, and 680.15 tokens. The repeat produced no empty answers. These descriptive measurements do not explain the cause of each changed score.

Keep the original fixed test intact

For the chat-trained model, the separate prescribed 128-question comparison remains **41/128 before and 80/128 after**, with 44 gains and five losses. The newly generated full-500 batch contains those same 128 questions, but its chat-format counts are 41 and 82. That later observation does not replace the original result. Changing evaluation batches can change generated answers.

In the original comparison, capped responses fell from 56 to 12. Of its 44 gains, 36 had a capped baseline response and 37 had no parseable baseline answer. Those categories overlap. They identify completion as an important limitation, but cannot distinguish changes in reasoning, repetition, and answer formatting on their own.

A chat prompt is extra input, not new weights

The official `qwen_math` option wraps each question in these role markers and adds a system instruction. The placeholder below stands for the unchanged dataset question [2].

```
<|im_start|>system
Please reason step by step, and put your final answer within \boxed{}.<|im_end|>
<|im_start|>user
[question]<|im_end|>
<|im_start|>assistant
```

The official `no` option passes just the question text. It adds no role markers and no instruction about reasoning or boxing the answer. Both formats still use the same model architecture, greedy decoding, 3000-token response limit, and final-answer checker. The question-only baseline demonstrates that this pretrained base model can already answer many questions without those extra instructions. Only the base model's input format has changed.

Which questions make up the full-500 result?

All 500 questions were excluded from our weight training. Their roles in this study nevertheless differ: 64 were repeatedly monitored, 128 formed the original fixed test, and 308 add further coverage. These counts come from the full-500 evaluations, with each trained checkpoint kept separate:

Input	Question role	Starting	Chat seed 42	Questions seed 42	Questions seed 43
Question only	Monitored 64	39	37	39	37
	Original test 128	79	84	84	81
	Additional 308	187	196	191	188
Chat prompt	Monitored 64	21	42	24	23
	Original test 128	41	82	45	40
	Additional 308	92	184	99	103

The additional questions broaden the evaluation, but the full set is not 500 entirely uninspected questions. More evaluation rows are not more training replicates. By the fresh training repeat, all 500 had already been evaluated. We do not know whether these problems appeared during the base model's pretraining.

Check the actual inputs, then repeat the result

The 10:35 question-only attempt was rejected: despite its command-line setting, a dataset cache supplied chat-formatted training inputs. The valid fresh run used an isolated cache, started at 10:40, and finished by 11:24. All 4096 saved training prompts were checked against the 512 raw source questions. A setting in a configuration file is not evidence of what the model actually received.

All five fixed-weight checks reproduced byte-identical saved answer files: question-only scores 305 (base), 314 (first question-trained), and 306 (fresh repeat), and chat scores 154 (base) and 308 (chat-trained). These are evaluation repeats, not additional training replications. All completed results are retained. Re-evaluation tests generation; retraining tests learning. The repeat's final monitoring score was 41/64, but the same 64 rows scored 37 in its full-500 batch. That generation difference does not change the reported 306/500 or justify selecting the better batch.

5 / A SMALL EXAMPLE

What did the pilot's two extra points mean?

Under the standalone checker, 19 monitoring answers stayed correct, 41 stayed incorrect, three became correct, and one became incorrect. Thus the net change is +2/64, or 3.125 percentage points. These are the earlier pilot's **monitoring results**, separate from the larger run's 41-to-80 test result.

Changed problem	What the saved responses show
Fuel use	Repetition without an answer became a completed calculation and boxed 550.
Magic square	The baseline already derived $n = 7$, then repeated an incorrect verification. The final response completed and boxed 7.
Recurrence	A boxed answer changed from 1089 to the correct 898. Both responses had valid answer formatting.
Trigonometry	A correct boxed 12 became repetitive noncompletion at the 3000-token limit.

A concrete example: getting an answer out reliably

Karla drove 12,000 miles. How much fuel would she save with a car averaging 48 miles per gallon instead of 15? The completed post-training response gives

$$\frac{12000}{15} = 800, \qquad \frac{12000}{48} = 250, \qquad 800 - 250 = \boxed{550}.$$

The baseline repeated a version of the question until the 3000-token limit. The final response completed in approximately 417 text tokens. This is useful behavioral improvement on that example, but it does not show that the model newly acquired division or subtraction.

The recurrence example is also instructive. Independent calculation gives $211 + 420 + 267 = 898$. Both responses wrote Python describing the correct recurrence, but the baseline's claimed code output was 1089. Those fenced "output" blocks were generated text; no Python tool had run. A correct final answer does not validate an entire explanation or prove successful tool use.

What the training evidence adds

The first collection had 12 rewarded responses out of 128, with seven mixed groups giving 56 nonzero advantages. Twenty responses reached the token limit. The four completed optimizer updates had finite nonzero gradients, sampled checkpoint tensors changed, and updated weights were sent back to the generator. This establishes an operational path from sampled answers to weight updates. It does not make the small monitoring gain statistically conclusive.

The pilot illustrates why an aggregate score needs accompanying examples. The larger chat-format gain and the modest question-only gains both need the same care in interpretation.

Sources and editable evidence.

[1] Liu et al., Understanding R1-Zero-Like Training: A Critical Perspective. https://arxiv.org/abs/2503.20783.

[2] Pinned official training, evaluation, and grading code; Oat package version.

[3] Pinned Qwen2.5-Math-1.5B model revision.

[4] Companion research record, data manifest, and resolved environment; main training receipt; fixed-128 scoring receipt; full-500 chat-format receipt; prompt-comparison receipt; question-only training receipt; question-trained final scores; fresh-repeat training receipt; fresh-repeat final scores; pilot receipt. The research record links the question-only answer files and scoring evidence.

APPENDIX / AN EXPLORATORY FOLLOW-UP

What if we use standard GRPO?

A separate fresh-base run using the authors' standard GRPO option scored **318/500** with question-only evaluation, versus **305/500** for the base: 24 answers became correct and 11 became incorrect. The corresponding Dr. GRPO question-trained models scored 314 and 306. All 500 GRPO answers were checked with the same full checker and against the source questions and references. The same GRPO weights scored **169/500** with the chat prompt, versus 154 for the base: 22 gains and seven losses. All 500 chat answers were likewise regraded and source-checked. There were 189 capped responses and no empty responses. Strong input-format dependence remains: 318 with the question alone versus 169 with the chat prompt.

This run used the same 512 training questions, eight sampled responses per question, 32 full-model BF16 updates, and learner seed 43. It restarted from the original base model, not from a Dr. GRPO checkpoint. The author actor's time-based sampling seed means the training randomness is not fully paired.

Only the official `critic_type` option changed, from `drgrpo` to `grpo`; the authors' source was not modified. That option jointly changes response-length averaging and reward-standard-deviation normalization. It also changes update scale, even with the same learning rate. This comparison cannot isolate either normalization's effect.

The question-only net gain is 13 answers, or 2.6 percentage points. It is an exploratory result on the already-inspected 500 questions, not evidence that GRPO is reliably better than Dr. GRPO. One training run cannot establish that superiority. The main lesson remains prompt sensitivity, a functioning training pipeline, and uncertainty about dependable gains from this short training budget [2,4].

APPENDIX / REFERENCE CHECK | 7 OCTOBER, 20:56 UTC

Testing the authors' model is not reproducing its training

We evaluated **the authors' released model**, Qwen2.5-Math-1.5B-Oat-Zero, at pinned revision `a98e47785407` (full identifier in the receipt). It answered **366/500 questions correctly (73.2%)**. The paper reports 74.2%, equivalent to 371 correct answers. Our result is five answers lower, or one percentage point. We have not established the reason for that difference.

We used the official Qwen-Math prompt, one greedy answer per question, the 3000-token response limit, and the full final-answer checker. Independent regrading matched every saved reward; all 500 ordered prompts and reference answers matched the official data. The reference receipt records the output, model and source hashes, generation settings, and inspected examples.

This is useful evidence that our evaluation can reach a similar score on the published weights. **We did not train this model.** Our short chat-trained model remains at 308/500; the released model's 366 is not our training gain. Nor does the checked score validate every reasoning step or prove an exact reproduction of the published evaluation. We retain the five-answer difference.

The larger retry was stopped after the teaching demonstration

The first attempt ran out of memory before completing a collection; inner updates are unconfirmed. The second failed during initialization with an incompatible allocator. The third completed 12 updates and sampled 13 collections, then ran out of memory computing a diagnostic statistic. It saved no checkpoint. These are runtime failures, not benchmark results.

Attempt 4 started at 16:02 UTC. A private copy of the learning package computes diagnostic entropy in smaller pieces and releases unused old prediction tensors. Three focused CPU tests passed, including matching actual learning-step updates in a fixture. A full-shaped GPU check also passed; 219 learning calls completed before the user-requested stop. The shared environment and official source are unchanged. Saved model and optimizer states reach update 216.

The retry starts from the original base weights, not a trained checkpoint. We selected 4096 of the authors' 8523 MATH level 3–5 questions with a fixed shuffle. No exact normalized question overlaps with MATH500 were found; paraphrases and pretraining exposure remain unassessed. It uses the authors' `math_verify` training verifier and chat prompt. Each collection has 16 questions and eight answers each, then one update and a weight transfer. Excluding two overlong prompts and dropping the incomplete batch gives **255 updates, 4080 questions, and 32,640 responses**. The earlier 3968-question/248-update large-collection plan was abandoned. These are budgets, not completed results.

Saving occurs every eight updates; training is capped at ten hours and the whole sequence at thirteen. The planned final checkpoint, `step_00256`, was not reached. MATH500 (500 questions), AMC (83), and Minerva (272) final tests in both input formats were canceled. Separate base-model and authors' model checks on the broader benchmarks are complete. They validate reference comparisons, not our learning; the companion research record keeps all four conditions and their counts.

What remains outside this demonstration

The question is whether this closer reference recipe improves on the strong untrained question-only baseline. Data coverage and difficulty, the verifier, and duration change together; a gain could not be attributed to longer training alone. MATH500 is now an inspected development benchmark, not independent confirmation. Repeated training and broader tests remain important. The original protocol fixed the endpoint; the original loader check records the excluded prompts. The stop receipt records why that endpoint was not reached.

A small-scale demonstration of learning

The longer run was stopped by choice after 219 updates. The table contains every completed monitoring result available at this cutoff. Each score counts correct final answers on the same 64 questions, with chat-style input, greedy generation, a 3000-token response limit, and the full answer checker.

Completed weight updates	0	32	64	96	128	160	192
Correct final answers out of 64	20	42	42	43	43	48	47

We independently regraded all 64 answers at each later point. Every result agreed with the saved score, and the questions, reference answers and input format matched the starting check. Relative to the start, update 192 gained 27 answers and lost none. Between 160 and 192, one answer improved and two regressed, lowering the score from 48 to 47. Progress is not monotonic. This small decline does not establish a worsening trend; retain every point, not just the best one.

What this does and does not show

The latest 47/64 is 73.44%, not a full MATH500 score. This is a small, repeatedly inspected development set from one training run. It does not yet show a reliable gain over the stronger question-only starting setup, a full MATH500 improvement, or better recursive reasoning. The old short model also scored 42 on these same questions when generated as part of its 500-question evaluation. The base model with questions alone scored 39 on that subset. Those batch settings differ from live monitoring, so these numbers are context, not interchangeable tests.

What we take forward to RLM experiments

The successful small-scale demonstration has served its purpose. Further math training and final evaluations are canceled; prepared math screens are shelved. Checkpoints and every completed score remain available. We did not select the highest-scoring point as a final result.

The next experiments ask whether an RLM can use its tools, produce a useful mixture of successful and unsuccessful attempts, and improve beyond its starting preparation. Compare prompt examples, verified demonstrations, teaching order, and exploration at matched budgets. Test new combinations of skills, not only the routines shown during teaching. Pages 11–15 develop these lessons.

An important scoring distinction

The earlier short-run native monitor used the faster training checker. Its native scores at baseline, update 32 and final bookkeeping step 33 were 19, 38, 35; rescoring those same answers with the full checker gives 20, 39, 36. The longer run uses the full checker already. We must not compare a native fast-checker score against a full-checker score without saying so. Final 500-question comparisons throughout this guide already use the same full checker.

The points 32, 64, 96, 128, 160 and 192 follow that many completed updates. The unreached final label 256 would have followed 255 completed updates. See intermediate evidence and the companion research record for raw-file hashes and the latest operational status.

TWO ACTUAL ANSWERS | 7 OCTOBER, 19:40 UTC

An answer can improve while another gets worse.

These examples come from the same 64-question check at updates 64 and 96. They illustrate changes in individual answers, not how often each kind of error occurs. We summarize the saved responses below; the arithmetic checks are ours.

An answer that improved: rolling two dice

Question: What is the chance that the product of two fair dice is a multiple of five?

At update 64, the model's final answer was 1/3. At update 96, it gave the correct answer, 11/36.

Why 11/36? At least one die must show five. There are six outcomes with five on the first die and six with five on the second. We counted $(5, 5)$ twice, so subtract one: $6 + 6 - 1 = 11$ favorable outcomes out of 36.

An answer that got worse: two integers

Question: If integers m, n satisfy $3m + 4n = 100$, what is the smallest possible value of $|m - n|$?

At update 64, the final answer was the correct value, 3. At update 96, the model wrote a search loop and claimed its output was $(m, n) = (28, 11)$, giving a difference of 17. But these numbers do not even satisfy the equation: $3(28) + 4(11) = 128$, not 100.

At update 160, this answer recovered to the correct value, 3. The earlier failure remains useful evidence; individual answers can improve, regress, and recover during training.

A simple check: Set $d = m - n$. Then $7n + 3d = 100$, so d must be one of $\ldots, -11, -4, 3, 10, \ldots$. The smallest absolute value is 3, attained by $(m, n) = (16, 13)$: $48 + 52 = 100$.

The lesson for our future RLM work

Writing code is not the same as running code. In this experiment, the model only generates text. Neither the search loop nor its claimed output was executed by our system. The answer checker marked the wrong final answer incorrect, but it does not supervise every reasoning step.

An RLM could actually execute a calculation and inspect the result. That is a different capability which needs its own tests. These examples motivate checking real tool returns, not treating a plausible-looking explanation as evidence that a tool was used correctly.

The evidence receipt identifies the saved responses (zero-based indices 11 and 57). No generated code was executed in this audit.

LESSONS FOR RLM TRAINING

Some right, some wrong: why practice must be within reach

The small reproduction gives us a useful starting result: the same 64 monitoring questions improved from 20 correct to 47. Its more important purpose is to help us design RL for a model that uses Python and delegates subtasks. The following ideas are proposed applications, not results of an RLM training experiment.

The contrast is within one question

Dr. GRPO subtracts the average reward for a question from each attempt's reward. If four attempts score $1, 1, 0, 0$, the average is 0.5, and their relative signals are $+0.5, +0.5, -0.5, -0.5$. If all eight attempts fail, or all eight succeed, every relative signal is zero. This refers to the binary group-relative reward term, not every possible RL algorithm or auxiliary loss.

An always-solved easy question and an always-failed hard question do not provide this contrast just because their combined accuracy is 50%. **Each question needs its own useful variation.** Nor does mixed success guarantee learning: the checker may be wrong, an answer may be lucky, or the update may assign credit to unhelpful parts of a long response.

Curriculum is a plausible way to keep practice useful

A curriculum could start with tasks the model sometimes solves, then gradually increase difficulty as those tasks become reliable. For an RLM, one progression might be reading a real Python result, making one valid helper call, passing a fact between two dependent calls, then choosing a decomposition without hints. This is a candidate curriculum; our math experiment has not tested it.

Pretraining and SFT are ways to supply a starting skill

We did not add SFT before math RL. The math-pretrained model could already produce correct answers. An RLM may know Python but not our interface: what arguments a helper accepts, what information it needs, and how results return. Clear examples might suffice. If not, SFT on verified demonstrations could teach those actions. A stronger model or teacher-generated examples are alternatives.

The goal is not to hand-code every solution. It is to make successful actions reachable so that training can improve choices among them. Keep a direct-answer option: unnecessary delegation should not count as success.

ONE CANDIDATE EXPERIMENT

Can preparation unlock RL of useful decomposition?

An example separates interface use from strategy

Suppose a biography names a scientist's employer, and another document gives the employer's city. A model might call a nonexistent helper function; that is an interface failure. It might make a valid call but omit the employer's name; that is a context-passing failure. A useful strategy identifies the employer, passes that name to the next lookup, and combines the returned evidence. This is an illustrative task, not a measured result from our current run.

More attempts and temperature are levers, not guarantees

Extra attempts can expose rare successes. With independent attempts each having a 1% success probability, the chance of any success is about 7.7% for eight attempts and 27.5% for 32. Real attempts may repeat the same error. Increasing temperature can diversify strategies but can also break syntax or tool calls. Spending more on one question leaves less compute for other questions.

Measure valid execution, final correctness, and the fractions of all-fail, mixed-success, and all-success groups. Do not confuse more evaluation attempts with a better trained model: keep the evaluation attempt budget fixed. If using partial rewards, verify that a model cannot earn them while avoiding the real task. More code, text, or helper calls are not achievements by themselves.

A small comparison before a large training campaign

Use the same model and tasks with three preparations: prompt examples alone, staged SFT on verified RLM demonstrations, and SFT on the same examples shuffled. Match SFT exposure and keep helper models fixed. This separates the value of demonstrations from their order. Sample eight attempts per training problem and diagnose where failures occur. Then give each setup the same RL budget and measure its own before/after gain on separate questions under the same tool and answer budget. Record preparation cost too; tune the design using training or development tasks.

If preparation helps but RL adds nothing, we have learned about instruction or SFT, not shown RL learning useful decomposition. If preparation creates useful alternatives and RL improves beyond it on new tasks, that supports the hypothesis. If contrast already exists but learning fails, investigate reward quality, credit assignment, and update behavior rather than adding SFT indiscriminately.

Working idea: Teach access to useful actions; explore alternatives; use verified outcomes to learn better choices. We have stopped the math run to focus on experiments that test this RLM question directly.

RELATED WORK

What the literature suggests we test

The human-learning analogy helps us imagine a curriculum, but does not establish which order works for a language model. These primary sources motivate narrower, testable questions. This is a targeted reading list, not an exhaustive novelty review.

DAPO: keep useful reward contrast

DAPO, section 3.2 filters all-correct and all-wrong response groups. It is a direct precedent for focusing updates on mixed outcomes. Adaptive filtering is not the same as teaching prerequisites in sequence. We should also account for the cost of discarded generations.

Recursive Language Models: train environment use

The RLM paper, version 3 describes fine-tuning Qwen3-8B on 1,000 filtered RLM demonstrations, with attention to root model use of the environment and recursive calls. It separately reports RL transfer from shorter to longer retrieval tasks. This directly motivates our direction, while ruling out a novelty claim for simply training an RLM.

DeepSeek-R1: supervised preparation is an option

DeepSeek-R1, section 2.3.1 adds a small supervised cold start before RL; R1-Zero does not. The lesson is to compare starting preparations, not assume every RL application requires added SFT.

STaR: grow a collection of successful examples

STaR iteratively generates and filters solution rationales for fine-tuning. An RLM adaptation could collect successful executed trajectories, then use them to improve the next round. Correct final answers alone would not certify every tool action or decomposition choice.

Our proposed contribution must be more specific

Our earlier research audit already records SFT learning executable routines, with weaker flexible transfer. The hypothesis is therefore about bridging local skills to whole-task behavior, not that we never taught basic tool use. Check existing competence first.

We could test whether a particular sequence of basic RLM competencies improves later RL more than the same examples shuffled, and whether the benefit transfers to new combinations of skills. First establish what the model already knows. Measure execution failures, reward contrast, and improvement beyond SFT alone. Keep a prompt-only baseline and a direct-answer option.

These papers motivate this experiment; they do not establish its outcome. The math run is stopped; the broader portfolio now guides our next RLM experiments.

POSSIBLE BLIND SPOTS

What else could be limiting RLM learning?

We should not settle too early on "we started too hard." Several explanations can produce the same low success rate, and they need different interventions.

Valid calls are not useful chains of calls

If a helper returns "Aster Lab," the next lookup must use that employer, not a memorized alternative. Change the returned name and check that the downstream query changes. Count correct information passing separately from valid syntax. Toolformer, section 7 explicitly reports that independently generated tool examples did not teach chained use.

A correct teacher may still be a poor teacher for this student

A strong model can produce a correct trajectory that a smaller learner cannot yet follow. Check short unaided continuations, not only teacher correctness. Informative Alignment finds that stronger teachers do not consistently produce better students in its math experiments. Applying that lesson to tool trajectories remains a hypothesis.

Perfect demonstrations do not cover the student's mistakes

Teach recovery from errors the learner actually reaches. Measure mistakes fixed and correct answers spoiled, with an equally costly independent retry as a control. SCoRe identifies this distribution mismatch in self-correction training; its result is not proof of RLM recovery.

A failed team does not identify the failed decision

The parent can choose a good subtask and receive a bad helper answer. Keep helpers fixed initially; later compare parent and helper snapshots separately. Dropping child tokens from the loss does not freeze a helper that shares changing weights. Token, turn and child-count weighting also need their own checks: our math run does not validate a recursive training objective automatically.

Feedback, reward and available information are different

Showing an error in the next observation helps the model know what happened. Scoring that error changes the training objective. Preserving an earlier return changes what information remains accessible. Our prior history-repair experiment already tested information availability; it was not an example-order curriculum. Do not rename a completed intervention as a new research question.

EXPERIMENT PORTFOLIO

Breadth with interpretable comparisons

Curriculum is one promising branch, not our conclusion. We can investigate several small questions using shared tasks, budgets, and failure measurements. These are proposals; no new RLM run has been launched from this discussion.

Preparation and practice

Compare a clear prompt with demonstrations, then protocol-only with dependent-call SFT. Separately compare the same SFT examples staged or shuffled. Before adding RL, measure what preparation alone achieved. During RL, compare uniform practice with selection of informative reward groups, including discarded generation cost.

Exploration and assistance

Compare more attempts on fewer questions with broader question coverage at the same total budget. Vary plans, not just wording. Test removing hints gradually against keeping them throughout training, and evaluate without hints. A training success supplied by a scaffold is not yet independent student competence.

Recovery, feedback and division of work

Compare successful-only teaching with error-and-repair examples. At saved helper boundaries, controlled correct and incorrect returns can diagnose whether the parent uses the information. Such oracle interventions are diagnostic conditions, not deployable scores. Keep final task success as the main evaluation criterion.

Generalization and simpler alternatives

Test familiar task structures, longer familiar structures, and new arrangements of dependent steps separately. Include tasks where direct answering is best. Compare a direct model, simple program, fixed decomposition and learned RLM under matched budgets. Useful delegation should earn its extra cost.

How we will use small experiments

First screen for informative differences; then train matched comparisons; then repeat promising effects independently. Preserve all outcomes, including failures. A small development set helps us choose experiments, not certify the best score among many trials. No universal optimal difficulty or teaching order is assumed.

Primary curriculum examples include E2H Reasoner (scheduling matters) and Self-Evolving Curriculum (adaptive category selection). These are related evidence, not our results. The living research portfolio on GitHub records the proposed controls and what would weaken each hypothesis.